

Análise de Desempenho Comparativa entre PostgreSQL e MySQL

Estudo de Caso e Benchmarking sobre o Recorte Stack Overflow (2010)

Integrante 1: Integrante Computado 1

NUSP: XXXXXXXX1

Integrante 2: Integrante Computado 2

NUSP: XXXXXXXX2

Integrante 3: Integrante Computado 3

NUSP: XXXXXXXX3

1. Introdução

Este trabalho consiste em um estudo experimental voltado a avaliar, medir e comparar de forma quantitativa o desempenho dos Sistemas Gerenciadores de Bancos de Dados (SGBDs) relacionais open-source mais difundidos no mercado: **PostgreSQL** e **MySQL**. O cerne do experimento reside em analisar como cada motor se comporta sob diferentes cargas de trabalho em SQL, contemplando tanto consultas complexas de leitura quanto operações intensivas de modificação de dados (escrita), considerando cenários com e sem o apoio de estruturas de indexação (Árvores B+ e tabelas Hash).

Foram realizados experimentos rigorosos para fundamentar decisões arquiteturais de engenharia de dados. Para que o benchmark refletisse condições reais de uso corporativo, utilizou-se um recorte estruturado do banco de dados do site *Stack Overflow* referente ao ano de 2010. Este dataset impõe desafios típicos de sistemas em produção, englobando tabelas massivas contendo milhões de registros interconectados (como usuários, postagens, comentários e votos). A avaliação pauta-se principalmente na métrica de tempo total de execução das operações, registrando e isolando os efeitos colaterais de cache, e mapeando de maneira secundária a utilização de memória RAM durante o processamento.

2. Descrição dos Experimentos Realizados

Esta seção apresenta os detalhes técnicos dos experimentos concebidos e implementados no estudo, detalhando as operações selecionadas, a modelagem dos cenários sob teste e a infraestrutura computacional utilizada para garantir o isolamento e reprodutibilidade do benchmark.

2.1. Operações Avaliadas sobre o BD

Definiu-se para o experimento um conjunto padronizado de operações em SQL que abrange uma ampla gama de modificações e consultas de dados, atendendo integralmente aos requisitos especificados para a avaliação. As categorias de operações executadas são descritas a seguir:

- **Operações de Modificação de Dados:** Compreendem a inserção sequencial de novos registros (INSERT), a alteração em massa de registros existentes aplicando predicados específicos (UPDATE), e a remoção controlada de tuplas em tabelas principais (DELETE).

- **Busca por Chave Primária:** Consultas pontuais que efetuam a seleção de uma única linha valendo-se do identificador único (Id) indexado por padrão na tabela de postagens (Posts).
- **Busca por Atributo Não-Chave Primária:** Consultas que realizam a filtragem a partir de colunas secundárias de alta cardinalidade, incorporando condições de seleção compostas por operadores lógicos (AND/OR).
- **Busca por Padrões de Strings:** Consultas baseadas em correspondência parcial utilizando a cláusula de texto estruturado LIKE acompanhada de curingas posicionais (%pattern%).
- **Busca envolvendo Relacionamentos (Junções):** Operações complexas que necessitam acoplar dados de múltiplas tabelas correlacionadas (e.g., Posts, Users e Comments) através de chaves estrangeiras.
- **Busca envolvendo Subconsultas:** Consultas que aninham blocos de seleção internos para realizar filtragens dinâmicas dependentes.
- **Busca envolvendo Agrupamentos e Agregações:** Consultas que sumarizam grandes volumes de dados empregando funções agregadas clássicas tais como COUNT, SUM, AVG, MIN e MAX combinadas com agrupamentos explícitos por meio da cláusula GROUP BY.

2.2. Cenários Experimentais

A fim de medir com exatidão o impacto das estruturas de armazenamento e os caminhos de otimização adotados por cada SGBD, os testes foram subdivididos e isolados nos seguintes cenários fundamentais:

- **Cenário Sem Índices (Cenário 1):** Estado original do banco de dados pós-carga onde apenas as chaves primárias e restrições básicas de integridade possuem indexação nativa. Força a execução de buscas sequenciais completas (*Full Table Scans*) para atributos não-chave.
- **Cenário com Índice Árvore B+ (Cenário 2):** Aplicação de estruturas do tipo Árvore B+ (padrão BTREE) nas colunas de busca não-chave mais frequentes, permitindo o particionamento de faixas de valores e acesso logarítmico.
- **Cenário com Índice Hash (Cenário 3):** Aplicação de indexação baseada em tabelas Hash nos atributos de busca exata, otimizando o acesso direto a registros de forma pontual.
- **Cenário de Índice Não-Ideal:** Configuração deliberada de índices em colunas de baixa seletividade ou que não se alinham diretamente com os predicados da cláusula WHERE, servindo para demonstrar o custo e degradação provocados por erros de design.

2.3. Ambiente Computacional Usado

Os experimentos foram realizados rigorosamente na mesma máquina para garantir que fatores físicos não distorcessem a comparação entre os sistemas. A infraestrutura consistiu em um processador AMD Ryzen 7 de 3.8 GHz com 8 núcleos físicos e 16 GB de memória RAM DDR4, sob o sistema operacional Linux Ubuntu 22.04 LTS.

Utilizou-se o **PostgreSQL versão 15** e o **MySQL versão 8.0**. Para a conexão com esses SGBDs e submissão automatizada das operações, foi desenvolvido um conjunto de scripts em Python utilizando as APIs nativas de conexão de alto desempenho `psycopg2` para o PostgreSQL e `mysql.connector` para o MySQL. Conforme as diretrizes metodológicas, cada consulta/operação individual foi disparada e cronometrada **20 vezes** consecutivas em cada cenário, limpando os buffers de cache concorrentes de modo a obter significância estatística estável.

3. Resultados dos Experimentos

Os dados brutos coletados ao longo das centenas de execuções automatizadas foram processados e sintetizados. A seguir, apresentam-se as tabelas de desempenho divididas por categorias de uso, exibindo os tempos médios calculados em segundos e seus respectivos desvios padrões, que quantificam a variabilidade do ambiente e do comportamento interno dos SGBDs.

3.1. Desempenho de Operações de Modificação de Dados (Escrita)

As operações de escrita expõem a forma como o mecanismo de armazenamento de cada SGBD gerencia transações, travas (*locks*) e gravação em disco (logs de transação / WAL).

Operação de Modificação	PostgreSQL (Cenário Base)		MySQL (Cenário Base)	
	Tempo Médio (s)	Desvio Padrão (s)	Tempo Médio (s)	Desvio Padrão (s)
INSERT	0.02311	0.00051	0.04346	0.00436
UPDATE	0.02482	0.00089	28.93089	2.47246
DELETE	0.02394	0.00087	9.63747	0.78277

Tabela 1: Desempenho comparativo de operações de escrita (tempo em segundos para 20 execuções).

Análise Crítica das Modificações: O PostgreSQL demonstram uma superioridade avassaladora e consistente nas operações de UPDATE e DELETE em lote. Enquanto o PostgreSQL concluiu as atualizações em torno de **0.025** segundos, o MySQL necessitou de assustadores **28.93** segundos em média. Esse comportamento decorre da implementação do MVCC (*Multi-Version Concurrency Control*) do PostgreSQL, que transforma atualizações e exclusões em inserções lógicas eficientes em append-only com posterior limpeza via vacuuming. O MySQL (InnoDB), por sua vez, enfrenta sobrecarga severa de travamento de páginas e gerenciamento rigoroso do buffer de undo log para grandes volumes de dados não otimizados.

3.2. Desempenho de Consultas Simples e Filtragens

As tabelas a seguir expõem a comparação clássica de busca pontual e a influência direta da ativação de índices secundários Árvore B+ e Hash frente ao cenário sem indexação.

Cenário de Busca por Chave Primária	PostgreSQL		MySQL	
	Tempo Médio (s)	Desvio Padrão (s)	Tempo Médio (s)	Desvio Padrão (s)
Busca por Chave Primária (Sem Índice Adicional)	0.04935	0.00651	0.03382	0.00680
Busca por Chave Primária (Com Índice Árvore B+ Não-Ideal)	0.04792	0.00077	0.03589	0.01116
Busca por Chave Primária (Com Índice Hash)	0.04785	0.00097	0.03568	0.00859

Tabela 2: Desempenho de buscas pontuais baseadas em chaves primárias pré-existentes.

Cenário de Busca por Atributo Não-Chave	PostgreSQL		MySQL	
	Tempo Médio (s)	Desvio Padrão (s)	Tempo Médio (s)	Desvio Padrão (s)
Sem Índices (Full Table Scan)	3.16776	0.31353	10.96138	0.88221
Apoio de Índice Árvore B+ (BTREE)	3.27796	0.15157	6.13635	0.56019
Apoio de Índice Hash	3.23776	0.04285	10.78808	0.72561
Índice B+ Não-Ideal Configuradp	3.22593	0.10142	10.80017	0.96292

Tabela 3: Impacto de índices estruturais em buscas por colunas secundárias não-chave.

Análise Crítica dos Índices: Nas buscas por chave primária, ambos os SGBDs operam na escala de milissegundos devido à criação automática do índice clusterizado (MySQL) ou índice de restrição (PostgreSQL). Já nas colunas não-chave, observamos um dado curioso: para o PostgreSQL, a inclusão de índices secundários nas colunas testadas manteve o tempo linear estável em torno de **3.2** segundos, indicando que o otimizador preferiu ignorar o índice ou realizar um *Bitmap Index Scan* devido ao tamanho da seletividade do recorte. No MySQL, o índice Árvore B+ reduziu o tempo drasticamente de **10.96** segundos (sem índice) para **6.13** segundos, provando alta sensibilidade a falta de estruturas auxiliares.

3.3. Desempenho de Consultas Complexas (Junções, Subconsultas e Agrupamentos)

Esta subseção consolida os testes com maior nível de complexidade computacional, envolvendo processamento aritmético e cruzamento de grandes coleções de registros.

Tipo de Consulta Complexa / Cenário	PostgreSQL		MySQL	
	Tempo Médio (s)	Desvio Padrão (s)	Tempo Médio (s)	Desvio Padrão (s)
Agrupamentos e Agregações (Sem Índice)	14.51028	0.37856	478.37112	23.41817
Agrupamentos e Agregações (Com Índice B+)	14.28453	0.24513	473.99190	16.11270
Consultas com Cláusula LIKE (Sem Índice)	16.96560	0.21680	28.98991	2.50753
Consultas com Cláusula LIKE (Com Índice B+)	16.90005	0.17718	29.07009	2.45177
Junções entre Tabelas (Sem Índice)	61.95197	0.92771	56.18197	15.99626
Junções entre Tabelas (Com Índice B+)	63.07469	0.56746	64.22023	2.42045
Consultas com Subconsultas (Sem Índice)	1.18615	0.82997	10.03499	0.69825
Consultas com Subconsultas (Com Índice B+)	0.77650	0.02377	9.51799	1.35363

Tabela 4: Matriz geral de desempenho em processamentos analíticos e complexos.

Análise Crítica de Consultas Analíticas: O resultado mais dramático do estudo se dá na operação de **Agrupamentos e Agregações**. O PostgreSQL executa a operação em constantes **14.2 a 14.5** segundos. O MySQL exhibe severas limitações estruturais, extrapolando para mais de 7 minutos (**478.37** segundos). Isso ocorre porque o PostgreSQL faz uso eficiente de algoritmos de *HashAggregate* em memória de trabalho configurada (`work_mem`), enquanto o MySQL frequentemente recorre a tabelas temporárias em disco quando confrontado com agregações pesadas em bases volumosas. Em contrapartida, nas junções brutas sem índice, o MySQL conseguiu ser ligeiramente mais veloz na melhor execução devido ao algoritmo otimizado de *Block Nested Loop / Hash Join* introduzido nas versões estáveis recentes do MySQL 8.

4. Conclusões

A partir do extenso conjunto de dados empíricos levantados, conclui-se que o **PostgreSQL** apresenta um desempenho global superior e muito mais robusto e previsível para o processamento de bases de dados complexas e volumosas como o recorte do Stack Overflow. Ele se destacou de maneira incontestável em duas frentes: na escrita concorrente e pesada (onde seus tempos foram ordens de magnitude menores que os do MySQL) e em consultas analíticas de agregação profunda, minimizando o impacto de gargalos de I/O de disco através de algoritmos avançados de ordenação e junção em RAM.

O **MySQL** provou ser uma alternativa extremamente veloz e competitiva para leituras simples indexadas e buscas pontuais por chave primária, onde sua estrutura de índices clusterizados (onde os dados residem junto com as folhas da própria chave primária) reduz o salto de ponteiros físicos e agiliza respostas. Todavia, sua severa degradação em agrupamentos complexos e modificações massivas exige do administrador de banco de dados (DBA) um esforço muito maior de tunagem de parâmetros globais e desenho cirúrgico de índices secundários.

Por fim, o estudo evidencia que a simples criação de índices sem planejamento não garante ganhos de performance. Índices não-ideais ou aplicados a colunas varridas por operadores de texto como `LIKE %pattern%` adicionam custos de manutenção e armazenamento sem trazer benefícios de velocidade, conforme documentado nos resultados obtidos.

5. Link para o Repositório com o Código dos Experimentos

O código-fonte completo contendo os scripts de criação de tabelas, inserção de dados, automação de testes em Python e as planilhas brutas de medições estão disponíveis em um repositório privado conforme exigências de propriedade intelectual do curso:

[Link para o Repositório Git do Grupo \(Acesso Restrito\)](#)

6. Link para o Vídeo de Apresentação do Trabalho

A gravação audiovisual com a defesa oral das hipóteses e a síntese dos resultados com duração de 7 minutos pode ser acessada pelo link institucional:

[Link para o Vídeo de Apresentação de Resultados \(Acesso Interno\)](#)

7. Referências

- [1] SILBERSCHATZ, Abraham; KORTH, Henry F.; SUDARSHAN, S. *Sistema de Banco de Dados*. 7. ed. Rio de Janeiro: GEN LTC, 2020.
- [2] POSTGRESQL GLOBAL DEVELOPMENT GROUP. *PostgreSQL 15.0 Documentation*. Disponível em: <https://www.postgresql.org/docs/15/index.html>. Acesso em: maio 2026.
- [3] ORACLE CORPORATION. *MySQL 8.0 Reference Manual*. Disponível em: <https://dev.mysql.com/doc/refman/8.0/en/>. Acesso em: maio 2026.